DEMO SESSION

Asynchronous Processing: Live Demo

From blocking HTTP to a job queue, on one machine

IF3110 – Web-based Application Development
Week 10 · Event-driven asynchronous processing

Where we left off

The lecture closed with three rules for slow work:

01

Accept early

Return 202 and a job ID. Finish the work later.

02

Decouple with messages

A broker sits between the producer and the consumer.

03

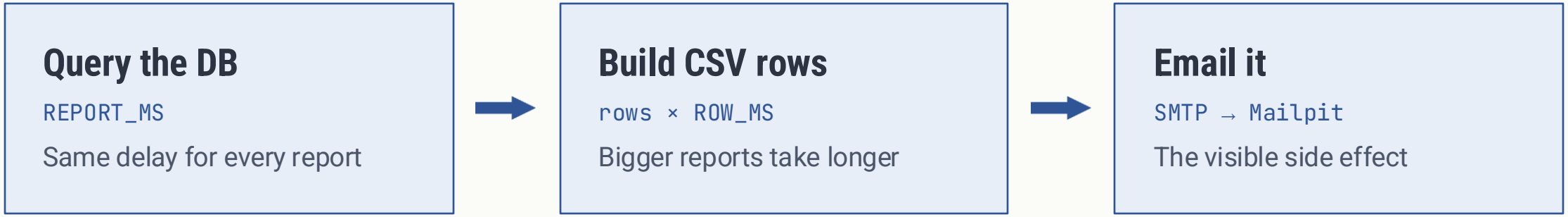
Design for retries

Failures are normal: retry, back off, park in a DLQ.

Today we watch each one happen on a real system.

The scenario: a slow report

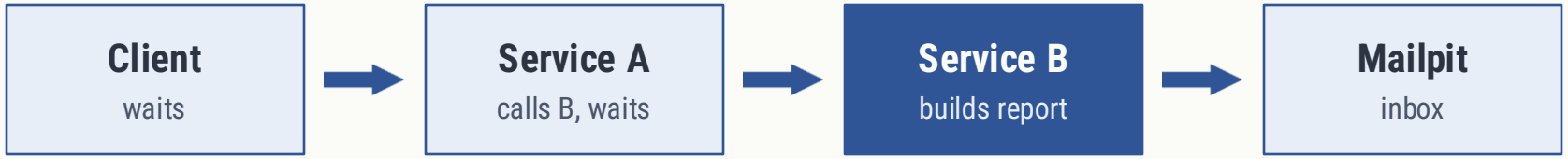
A user clicks **Generate report**. The system builds a CSV and emails it. Each report runs three steps:



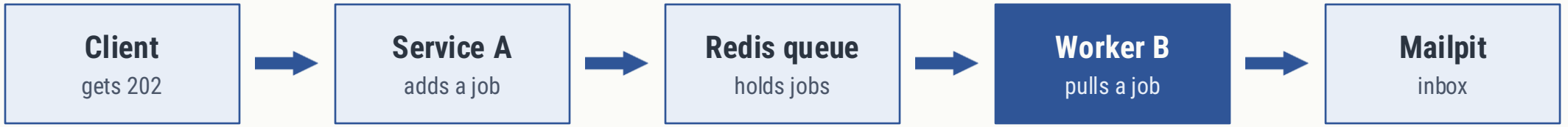
What happens when 30 users click “Generate” at once?

Two architectures, same capacity

SCENARIO 1 · SYNC (BLOCKING)



SCENARIO 2 · QUEUE



> **Only B is limited** (the dark boxes): **WORKER_REPLICAS × WORKER_CONCURRENCY** slots, identical in both scenarios. A has no limit. The only thing that changes is the architecture.

Get the demo

DEMO REPOSITORY

github.com/bayusamudra014/queue-simulation

1 Clone it

```
git clone https://github.com/bayusamudra014/queue-simulation
cd queue-simulation
```

2 You need

Docker with Compose, and Node 18+ for the two scripts (no npm install)

3 Check it starts

```
docker compose --profile sync up --build
```

What's inside

Path	Role
sync/	Scenario 1: A waits for B
queue/	Scenario 2: A, worker, SSE
shared/	The report job and email
load-test.js	Simulated users
watch-stats.js	Live queue view
.env	Settings for each run

Your instruments

Settings we will change (.env)

Variable	Today	What it controls
WORKER_REPLICAS	2	Number of Containers of Service B
WORKER_CONCURRENCY	2	Reports per container at once
REPORT_MS	1000	Simulated DB query time (ms)
ROW_MS	1	Time to build one row (ms)
SUCCESS_RATE	1	Chance one attempt succeeds
DLQ_ENABLED	false	Retries + dead-letter queue
JOB_ATTEMPTS	3	Tries before the DLQ

> **Service B capacity = WORKER_REPLICAS × WORKER_CONCURRENCY**
>
> Override for one run: WORKER_REPLICAS=3 docker compose --profile queue up

Scripts

load-test.js · simulated users

```
node load-test.js <baseUrl>
<total> <perSecond> [rows]
```

total: how many users · **perSecond**: arrival rate · **rows**: report size, e.g. 500 or 10-5000

Prints a table of status codes. In queue mode it follows each job until its email is sent.

watch-stats.js · live queue view

```
node watch-stats.js
http://localhost:3000
```

Every second: waiting / active / completed / failed / dlq, plus a bar for the queue length. Queue mode only.

Every email lands in Mailpit: http://localhost:8025 · nothing reaches real addresses

Settings for each experiment

Experiment	Profile	.env values (changes from the baseline)
Baseline	–	WORKER_REPLICAS=2 · WORKER_CONCURRENCY=2 · SUCCESS_RATE=1 · DLQ_ENABLED=false
1 · Blocking	sync	Baseline
2 · Queue, same load	queue	Baseline
3 · Watch one job	queue	REPORT_MS=20000
4A · Scale out	queue	WORKER_REPLICAS=3
4B · Stop the worker	queue	Baseline
5.1 · Sync failures	sync	SUCCESS_RATE=0.3
5.2 · Queue, no retry	queue	SUCCESS_RATE=0.3
5.3 · Queue + DLQ	queue	SUCCESS_RATE=0.3 · DLQ_ENABLED=true · JOB_ATTEMPTS=3

> **REPORT_MS and ROW_MS never change**, so every run does the same amount of work and the results stay comparable.

> **After editing .env**, restart: docker compose --profile <p> down, then up. Or prefix one run: WORKER_REPLICAS=3 docker compose …

EXPERIMENT 1 · RUN

Blocking: the client waits

1 Start the sync scenario

```
docker compose --profile sync up --build
```

2 Send one request and feel the wait

```
curl -X POST localhost:3000/reports \
-H "content-type: application/json" \
-d '{"email":"me@example.com"}'
```

3 30 users arrive, 10 per second

```
node load-test.js localhost:3000 30 10
```

Predict

Of 30 users, how many get **200**?
What happens to the rest?

Your prediction:

Lecture link: “When HTTP waits” (slide 4)

EXPERIMENT 1 · THINK

Service B has free slots again after each report. Why are most users refused instead of waiting their turn?

> Follow Up: Would adding more B replicas fix this?

Discuss in pairs · 2 min

Queue: accept now, finish later

1 Switch to the queue scenario

```
docker compose --profile sync down
docker compose --profile queue up --build
```

2 Terminal 2: watch the queue

```
node watch-stats.js http://localhost:3000
```

3 Terminal 3: the same load as before

```
node load-test.js localhost:3000 30 10
```

Predict

How many get **202**? Do all reports eventually arrive? Who waits longest?

Your prediction:

Lecture link: “Accept now, finish later” (slide 5) · “The broker” (slide 17)

EXPERIMENT 2 · THINK

Service B’s capacity didn’t change. Where did the requests that were refused before go?

> Why did early users get their email quickly while later users waited much longer? Is the user better off, and what did we trade?

Discuss in pairs · 2 min

EXPERIMENT 3 · RUN

Watch one job: 202, then events

1 Send one report: note the jobId and eventsUrl

```
curl -X POST localhost:3000/reports \
-H "content-type: application/json" \
-d '{"email":"me@example.com"}'
```

2 Follow its events (Server-Sent Events)

```
curl -N localhost:3000/reports/<jobId>/events
```

-N = no buffering: print each event as it arrives

3 Open the inbox

```
http://localhost:8025
```

Predict

When does the email arrive, compared with the 202? Which events will the stream show?

Your prediction:

Lecture link: “HTTP 202 Accepted” (slide 9) · “Job status lifecycle” (slide 10)

EXPERIMENT 3 · THINK

When the client receives 202, is the report done? Which component is “correct” at which moment?

> Follow-up: the HTTP response already ended. How does the client still get updates: polling, SSE or a webhook? Which would you choose?

Discuss in pairs · 2 min

EXPERIMENT 4 · RUN

Scale out, then pull the plug

A Three workers, same load

```
WORKER_REPLICAS=3 docker compose --profile queue up
node load-test.js localhost:3000 30 10
```

B Stop the worker in the middle of a test

```
node load-test.js localhost:3000 30 10
docker compose stop worker-b
# watch the queue for a while, then:
docker compose start worker-b
```

Predict

How does the drain speed change with more workers? What happens to new requests while no worker runs?

Your prediction:

Lecture link: “More workers, same queue” (slide 19)

EXPERIMENT 4 · THINK

Why can we add workers freely, while adding sync B replicas didn’t fix the spike?

> Follow-up: with the worker stopped, why did the API keep answering 202?
>
> Follow-up: with 3 workers, are jobs finished in submission order? Does it matter here?

Discuss in pairs · 2 min

EXPERIMENT 5 · RUN

Failures, retries and the dead-letter queue

Each attempt now succeeds only 30% of the time. Restart in each mode, then run the same load test.

1 Sync

```
SUCCESS_RATE=0.3 docker compose --profile sync up
```

2 Queue, no retry

```
SUCCESS_RATE=0.3 docker compose --profile queue up
```

3 Queue with retries and DLQ

```
SUCCESS_RATE=0.3 DLQ_ENABLED=true JOB_ATTEMPTS=3 \
docker compose --profile queue up
```

→ Inspect and replay dead letters

```
curl localhost:3000/dlq
curl -X POST localhost:3000/dlq/<id>/retry
```

Predict

In each mode, what happens to a failed report? With retries on, will more succeed? Will all of them?

Your prediction:

Lecture link: “Retry and backoff” (slide 26) · “Dead-letter queue” (slide 28)

EXPERIMENT 5 · THINK

Why park failed jobs in a dead-letter queue instead of retrying forever?

> Follow-up: a job fails *after* the email was already sent, and gets retried. What could go wrong?
>
> Bonus: how would you estimate the success rate from SUCCESS_RATE and JOB_ATTEMPTS?

Discuss in pairs · 2 min

Connecting the dots

Experiment	What we saw	Lecture concept
1 Blocking	Only a few succeed, most get 503	HTTP waiting, capacity pressure
2 Queue	Everyone gets 202, finishing over time	Broker as buffer, 202 Accepted
3 One job	202 long before the email	Job lifecycle, eventual consistency
4 Scale / stop	Faster drain, nothing lost	Competing consumers, decoupling
5 Failures	Retries recover many, the rest go to the DLQ	Retry and backoff, DLQ, idempotency

Queues need limits too

Send a very fast burst and the queue just grows. Cap its length or rate-limit the producer.

When is a queue overkill?

When the work is fast and the user needs the answer immediately.

Exit ticket

BEFORE YOU LEAVE

Name one step in this demo that must be idempotent, and explain why.

Try it at home

Clone the demo repo (slide 5), then run both scenarios:

```
docker compose --profile sync up --build # scenario 1
docker compose --profile queue up --build # scenario 2
node load-test.js localhost:3000 30 10
```